SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	B V Chaitantya Reddy	**Reg. No.:**	192425376
Section / Batch:	AIML	**Date:**	

Code of Conduct

I ___B V Chaitanya_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____chaitanya_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided

Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	3	3	3	3	3	15	83
2	3	3	3	3	3	15	
3	4	3	4	4	4	18	
4	3	4	3	4	4	20	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Code:

```
# HMM POS Tagging using basic Python

corpus = [
    [("The","DT"), ("boy","NN"), ("eats","VBZ"), ("rice","NN")],
    [("The","DT"), ("girl","NN"), ("drinks","VBZ"), ("milk","NN")],
    [("A","DT"), ("cat","NN"), ("drinks","VBZ"), ("milk","NN")],
    [("The","DT"), ("dog","NN"), ("chases","VBZ"), ("cat","NN")],
    [("A","DT"), ("teacher","NN"), ("teaches","VBZ"), ("students","NNS")],
    [("Students","NNS"), ("study","VBP"), ("English","NN")],
    [("Birds","NNS"), ("fly","VBP"), ("high","RB")],
    [("Children","NNS"), ("play","VBP"), ("games","NNS")]
]

# Count emissions and transitions
emission = {}
tag_count = {}
transition = {}

for sentence in corpus:
    previous = None

    for word, tag in sentence:

        tag_count[tag] = tag_count.get(tag, 0) + 1

        if tag not in emission:
            emission[tag] = {}
```

```
        emission[tag][word] = emission[tag].get(word, 0) + 1

        if previous:
            if previous not in transition:
                transition[previous] = {}

            transition[previous][tag] = \
                transition[previous].get(tag, 0) + 1

        previous = tag


# Convert counts to probabilities
for tag in emission:
    for word in emission[tag]:
        emission[tag][word] /= tag_count[tag]

for tag in transition:
    total = sum(transition[tag].values())

    for next_tag in transition[tag]:
        transition[tag][next_tag] /= total


# Display emission probabilities
print("EMISSION PROBABILITIES")
print("-" * 50)

for tag in emission:
    for word, prob in emission[tag].items():
        print(tag, word, round(prob, 3))


# Display transition probabilities
print("\nTRANSITION PROBABILITIES")
print("-" * 50)

for tag in transition:
    for next_tag, prob in transition[tag].items():
        print(tag, "->", next_tag, round(prob, 3))


# Viterbi algorithm
def viterbi(sentence):

    tags = list(emission.keys())
    dp = [{}]
    path = {}

    # First word
    word = sentence[0]
```

```
    for tag in tags:
        dp[0][tag] = emission[tag].get(word, 0.0001)

        path[tag] = [tag]

    # Remaining words
    for i in range(1, len(sentence)):

        word = sentence[i]
        new_dp = {}
        new_path = {}

        for current in tags:

            best_prob = 0
            best_previous = None

            emit = emission[current].get(word, 0.0001)

            for previous in tags:

                trans = transition.get(
                    previous, {}
                ).get(current, 0.0001)

                prob = dp[i-1][previous] * trans * emit

                if prob > best_prob:
                    best_prob = prob
                    best_previous = previous

            new_dp[current] = best_prob
            new_path[current] = path[best_previous] + [current]

        dp.append(new_dp)
        path = new_path

    best_tag = max(dp[-1], key=dp[-1].get)

    return path[best_tag]


sentence = ["The", "cat", "drinks", "milk"]

result = viterbi(sentence)

print("\nVITERBI RESULT")
print("-" * 50)

for word, tag in zip(sentence, result):
    print(word, "->", tag)
```

output:

```
NN English 0.1
VBZ eats 0.2
VBZ drinks 0.4
VBZ chases 0.2
VBZ teaches 0.2
NNS students 0.2
NNS Students 0.2
NNS Birds 0.2
NNS Children 0.2
NNS games 0.2
VBP study 0.333
VBP fly 0.333
VBP play 0.333
RB high 1.0

TRANSITION PROBABILITIES
--------------------------------------------------
DT -> NN 1.0
NN -> VBZ 1.0
VBZ -> NN 0.8
VBZ -> NNS 0.2
NNS -> VBP 1.0
VBP -> NN 0.333
VBP -> RB 0.333
VBP -> NNS 0.333

VITERBI RESULT
--------------------------------------------------
The -> DT
cat -> NN
drinks -> VBZ
milk -> NN
```